pyAmpliCol at one loop

Born–virtual interference: status, benchmarks and architecture

Report 2026-10-20 · timings 2026-10-18 · AMD EPYC 9754, one pinned thread

pyAmpliCol computes the interference $2\,\mathrm{Re}\left(B^{\dagger}GL\right)$ of one-loop and Born amplitudes with the UV and R_2 counterterms supplied by the UFO loop model, returning all three Laurent coefficients for every squared coupling order in binary64, double-double (DD) and arbitrary precision (AP). This report tracks the required $2 \to 5$ processes, the measured cost of the processes completed so far, and the architecture that produced them. It marks what is not measured, not validated or not working as such.

Status of the timing data. All runtimes were measured on 2026-10-18, at revisions fa599885, f89249d2 ($u\bar{u} \to ggg$) and 85fb0900 (EW). They time the *pre-shared-weight* engine through its Python coordinator with JIT coefficient evaluators, including exact decimal input, on repeated three-point streams. The delivered runtime (artifacts, CLI, SDKs) instead runs the interpreted evaluator in the Rust executor, which is not timed here. These are development observations, not the frozen acceptance campaign; later features have not been retimed (protocol box in Section 2).

Acceptance gates (on the required $2 \to 5$ set; development rows for orientation)

Required processes validated	**0 / 8**
Required processes timed	**0 / 8**
$t < t_{\mathrm{MadLoop}}$, **development rows**	**0 / 12 · ×2.0–×14.8**
$t \le 1.3t_{\mathrm{Recola}}$, **development rows**	**11 / 14 · ×0.1–×2.4**
$t \le 1.3t_{\mathrm{OpenLoops}}$, **development rows**	**2 / 10 · ×0.9–×5.3**

Counts are process × batch cells at $B = 1$ and $B = 128$ (Section 2).

1 Required processes and feasibility

The development plan requires the eight $2 \to 5$ processes of Table 1, to be compared with MadLoop and Recola, and with OpenLoops where a public library exists. None has a result yet. The seven-leg control $u\bar{u} \to d\bar{d}s\bar{s}g$ measures where generation spends its time (Section 4).

Table 1: **Required $2 \to 5$ processes** (plan §5.7) and the seven-leg control. Graph counts from MadLoop or pyAmpliCol generation as marked; — means not counted. *No result*: no complete generation exists. *Blocked*: a known model-admission gap stops generation. Sources in Section 16.

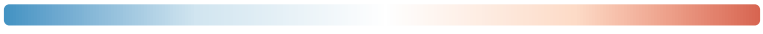
Process	Corr.	Born	Loops	Generation	Valid.	Timed	References	Main blocker
$gg \to gggg$ OL ppjjjj / gggggg	QCD	220	26 975 MadLoop	no result counterterm probe: over 8 GB within 165 s	no	no	Recola, OpenLoops	all scaling walls; needs a colour basis and shared recursion
$gg \to t\bar{t}gg$ OL pptt jj / ttxgggg	QCD	123	7 356 source unstated	no result	no	no	MadLoop, Recola, OpenLoops	the scaling walls; massive-quark counterterms are validated on $gg \to t\bar{t}$
$u\bar{u} \to e^{-}e^{+}ggg$ OL pplljjj / eexuuxggg	QCD	100	5 538 MadLoop	no result	no	no	MadLoop, Recola, OpenLoops	symbolic projectors, counterterm enumeration, colour overlaps
$u\bar{u} \to e^{-}e^{+}d\bar{d}g$ OL pplljjj / eexuuxddxg	QCD	48	1 668 MadLoop	no result	no	no	MadLoop, Recola, OpenLoops	symbolic projectors, counterterm enumeration, colour overlaps
$gg \to t\bar{t}e^{-}e^{+}g$ OL pplltt j / eexttxggg	QCD	—	—	no result	no	no	MadLoop, Recola, OpenLoops	the scaling walls; massive-quark counterterms are validated on $gg \to t\bar{t}$
$gg \to t\bar{t}e^{-}e^{+}g$ OL pplltt j_ew / eexttxggg	EW	—	—	blocked 1351 of 1918 G_{μ} counterterm bases uncertified; CMS not wired in	no	no	MadLoop, Recola, OpenLoops	generic EW counterterms, CMS, joint QCD+EW selection
$gg \to ggggg$ no public OL library	QCD	2 485	—	no result formal $\mathrm{SU}(N_c)$ colour fails on 14 of 24 seven-gluon pairs	no	no	none generated	needs a colour basis and shared recursion
$u\bar{u} \to ggggg$ no public OL library	QCD	1 240	—	no result MadLoop graph count unfinished after 285 s	no	no	none generated	needs a colour basis and shared recursion
Control at seven legs								
$u\bar{u} \to d\bar{d}s\bar{s}g$ no public OL library	QCD	64	3 540 pyAmpliCol	stopped after 9.43 h, inside the UV/R_2 projector; peak RSS 77.2 GiB	no	no	MadLoop, Recola (3 exact points)	UV/R_2 projector wall; the run's source predates the Bose-alias counterterm fix 41ccde87

2 Runtime of completed processes

Table 2 compares the complete binary64 evaluation (Born, all helicities, full colour, UV, model R_2, all three Laurent coefficients) of the seven timed processes with MadLoop, Recola and OpenLoops at $B = 1$ and $B = 128$ points per call.

Table 2: **Runtime per phase-space point**, µs, binary64, full colour. Bold: median of five warm calls; below: min–max of the five. Multiplier: time ÷ the fastest timed reference for that process and batch (marked *fastest*), shown and coloured only when MadLoop is timed for that row; the EW row gives its ratio to Recola uncoloured. Grey lines: generation (pyAmpliCol: fresh process to complete plan, with peak RSS; MadLoop: export and static compilation; Recola: parameters, process definition and generation) or OpenLoops setup of a prebuilt library (source generation not timed). Recola Drell–Yan is matched photon-only. B: points per call.

Process	B	pyAmpliCol	MadLoop	Recola	OpenLoops
$u\bar{u} \to e^-e^+$ photon only · 1 loop · 1 family	1	**584** ×12.5 406–687	**46.9** fastest 46.7–48.8	**1 880** ×40.1 1 860–1 980	no matched control
	128	**147** ×3.2 128–235	**46.5** fastest 46.3–47.2	**1 780** ×38.4 1 780–1 800	no matched control
	gen.	2.43 s · 132 MiB	198 s	0.135 s	—
$u\bar{u} \to d\bar{d}$ QCD · 14 loops · 8 families	1	**1 730** ×6.9 1 630–1 830	**251** fastest 250–265	**2 650** ×10.5 2 630–2 780	**548** ×2.2 529–596
	128	**764** ×3.0 735–813	**254** fastest 253–263	**2 670** ×10.5 2 570–2 720	**563** ×2.2 556–671
	gen.	7.09 s · 180 MiB	14.4 s	0.253 s	setup 0.0382 s
$u\bar{u} \to t\bar{t}$ QCD · 14 loops · 9 families	1	**2 010** ×5.7 1 940–2 070	**352** fastest 319–434	**2 910** ×8.3 2 880–3 180	**655** ×1.9 634–710
	128	**745** ×2.0 726–1 000	**376** fastest 373–382	**3 050** ×8.1 2 950–3 300	**686** ×1.8 671–699
	gen.	6.39 s · 183 MiB	17.1 s	1.12 s	setup 0.0103 s
$u\bar{u} \to gg$ QCD · 45 loops · 20 families	1	**6 830** ×14.8 6 730–7 150	**463** fastest 381–494	**5 340** ×11.5 4 830–5 670	**1 700** ×3.7 1 550–1 860
	128	**3 100** ×8.3 2 780–3 380	**372** fastest 371–374	**4 470** ×12.0 4 200–5 360	**1 700** ×4.6 1 640–1 760
	gen.	21.2 s · 321 MiB	29.1 s	0.183 s	setup 0.0149 s
$gg \to gg$ QCD · 186 loops · 52 families	1	**33 700** ×13.0 31 000–33 800	**2 590** fastest 2 460–2 840	**36 100** ×13.9 33 500–43 600	**11 900** ×4.6 11 500–12 700
	128	**14 300** ×6.9 13 500–14 600	**2 070** fastest 2 070–2 090	**33 200** ×16.0 32 000–35 300	**16 200** ×7.8 13 300–17 700
	gen.	98.7 s · 1.06 GiB	30.1 s	0.170 s	setup 0.0130 s
$u\bar{u} \to ggg$ QCD · 556 loops · 119 families	1	**194 000** ×5.4 180 000–240 000	**36 100** fastest 32 100–36 800	**81 500** ×2.3 80 700–84 600	**36 900** ×1.0 36 200–56 300
	128	**125 000** ×3.7 122 000–131 000	**33 500** fastest 32 800–37 100	**88 600** ×2.6 85 400–88 700	**43 400** ×1.3 40 700–50 300
	gen.	863 s · 5.20 GiB	23.9 s	0.724 s	setup 0.0109 s
$e^-e^+ \to \mu^-\mu^+$ full EW, G_μ · 98 loops · 24 families	1	**6 590** ×2.1 6 470–6 740 vs Recola only	validated, not timed	**3 100** fastest 3 050–3 150	not timed
	128	**2 920** ×1.0 2 850–3 030 vs Recola only	validated, not timed	**2 950** fastest 2 940–2 980	not timed
	gen.	116 s · 711 MiB	—	0.175 s	—



×1/16 faster ×1/4 ×1 ×4 time ÷ fastest reference, log scale, clamped ×16 slower

Six further processes pass their complete physics checks but have not been timed (Table 3); validation durations are not used as runtimes.

Table 3: **Validated, not yet timed.** Complete components agree with the listed references in every listed arithmetic.

Process	Scope	Born / loops / fam.	Arithmetic	References	Runtime
$u\bar{u} \to e^-e^+gg$	full γ/Z	16 / 528 / 71	double, DD, AP80	MadLoop, Recola	not timed
$u\bar{u} \to d\bar{d}s\bar{s}$	eight six-propagator families	7 / 275 / 94	double, DD, AP80	MadLoop, Recola	not timed
$u\bar{u} \to d\bar{d}$	full CKM, mixed Born orders	—	double, DD, AP70	MadLoop	not timed
$u\bar{d} \to W^+g$	stable real-mass W; also $dW^+ \to ug$ and $d\bar{u} \to W^-g$	—	double, DD, AP70	MadLoop, Recola	not timed
$e^-e^+ \to u\bar{u}$	full EW, real masses, G_μ	2 / 99 / 24	double, DD, AP70	MadLoop, Recola	not timed
$u\bar{u} \to \sigma_8\sigma_8 g$	scalar QCD, isolated dependency candidates	6 / 201 / 58	double, DD, AP60/80	MadLoop	not timed

Measurement protocol and deviations from the plan's timing protocol

- CPU 382 (AMD EPYC 9754 · one pinned CPU thread · shared host); one thread per process.
- Five warm calls per batch after warm-up; medians with min–max. No paired blocks or uncertainty estimate (protocol item 9).
- *Repeated* three-point streams, not distinct frozen points (item 1).
- pyAmpliCol includes Python batch scheduling and exact decimal ingress (item 3); providers start from binary64 arrays. The delivered runtime and SDKs use the interpreted evaluator instead.
- pyAmpliCol rescue off; providers keep their stability checks, rescue and caches (item 6). OpenLoops actual-pole mode reruns the amplitude three times.
- Development-profile native builds, not release builds. DD/AP not timed. A release A/B (2026-10-20, not retimed here) was faster by ×5.9 for native $gg \to gg$ at $B = 32$, by ×5.3–×5.7 for colour overlaps and by ×3.5–×4.0 for generation.
- Repeat runs drift: an audit at $B = 128$ gave OpenLoops $gg \to gg$ 13 900 µs and MadLoop 2 670 µs, against 16 200 and 2 070 µs in Table 2. Ratios are therefore given to one decimal.

3 Where the time goes

The instrumented runs split each evaluation into exclusive clocks (Table 4, Figure 1). Reduction and scalar masters are a few per cent of the total; the per-graph numerators, the Born weighting and the counterterm currents dominate, followed by work no clock attributes.

Table 4: **Stage decomposition**, µs per point at the batch B of the instrumented run, which is separate from the warm runs of Table 2. pyAmpliCol: numerator = loop-coefficient DAGs, reducer callbacks and coefficient-head preparation; Born weighting = dual Born weights; counterterms = UV/R_2 and external-field scalars, coefficients and projection; NinjaRS = exclusive reduction; masters = oneloopmaster adapter; other clocked = parameters, Born coefficients and projection, family masses and assembly; unattributed = clocked total minus all stages (external states, kinematics, coordinator and timer overhead). Medians of per-sample values. MadLoop (grey rows): loop coefficients, shared HELAS (Born, local UV/R_2, external fields, wavefunctions), reduction plus masters, and residual; its stage clocks exclude the shared HELAS work from the numerator.

Process	B	Numerator	Born weighting	Counter-terms	NinjaRS	Masters	Other clocked	Unattrib.	Total	Red. +mast.
$u\bar{u} \to e^-e^+$	32	24.7	1.85	2.98	0.919	0.347	3.23	63.6	96.8	1.3 %
		6.47	—	0.651	35.9		—	10.0	52.9	68 %
$u\bar{u} \to d\bar{d}$	32	294	30.6	82.9	7.76	9.05	4.55	265	691	2.4 %
		55.8	—	1.87	197		—	11.8	266	74 %
$u\bar{u} \to t\bar{t}$	32	335	35.7	95.1	16.1	18.3	5.47	385	890	3.9 %
		94.5	—	3.59	257		—	16.2	371	69 %
$u\bar{u} \to gg$	32	1 780	337	263	21.5	17.7	15.0	764	3 200	1.2 %
		205	—	8.21	127		—	40.0	380	33 %
$gg \to gg$	32	9 190	1 920	654	88.4	68.8	24.7	1 890	14 200	1.1 %
		1 920	—	58.9	825		—	276	3 080	27 %
$u\bar{u} \to ggg$	128	55 500	46 300	13 500	268	288	207	13 600	130 000	0.4 %
		26 400	—	689	2 470		—	6 040	35 600	7 %
$e^-e^+ \to \mu^-\mu^+$	128	1 540	135	287	31.3	67.0	7.25	854	2 930	3.4 %
									MadLoop: validated, not timed	

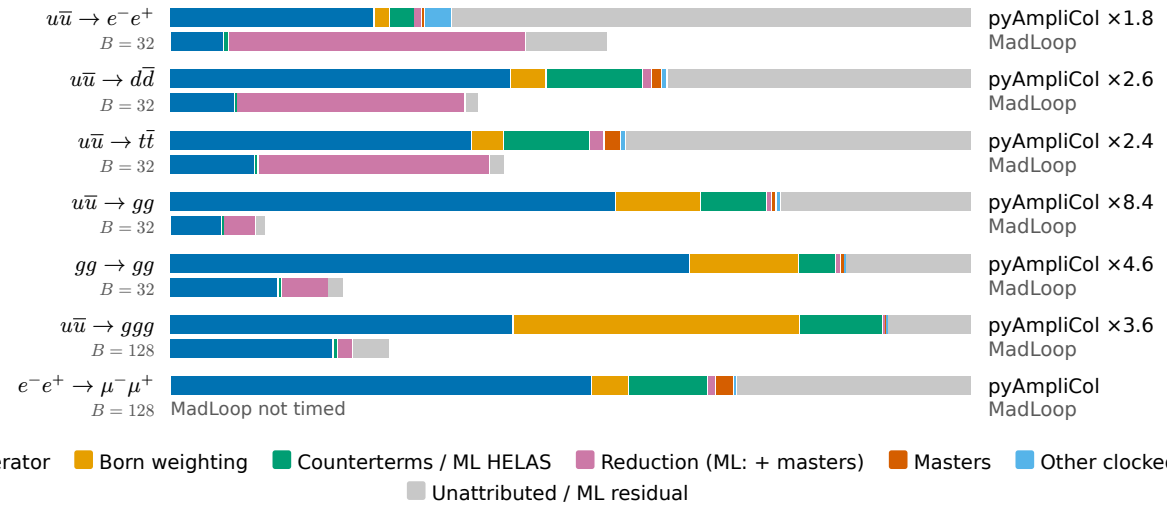


Figure 1: **Composition of one evaluation**, pyAmpliCol (upper bar) and MadLoop (lower bar) at the batch of Table 4, to the scale of the larger clocked total. Each bar ends at its median total; the grey end is the total minus the drawn stage medians. The label is the ratio of the totals. The thin purple and vermilion slivers are all of reduction and masters.

4 Generation cost and scaling

Generation is the present blocker for the required processes. Its time and memory grow faster than linearly with the number of loop graphs (Figure 2), and the seven-leg control shows which stages dominate (Figure 3).

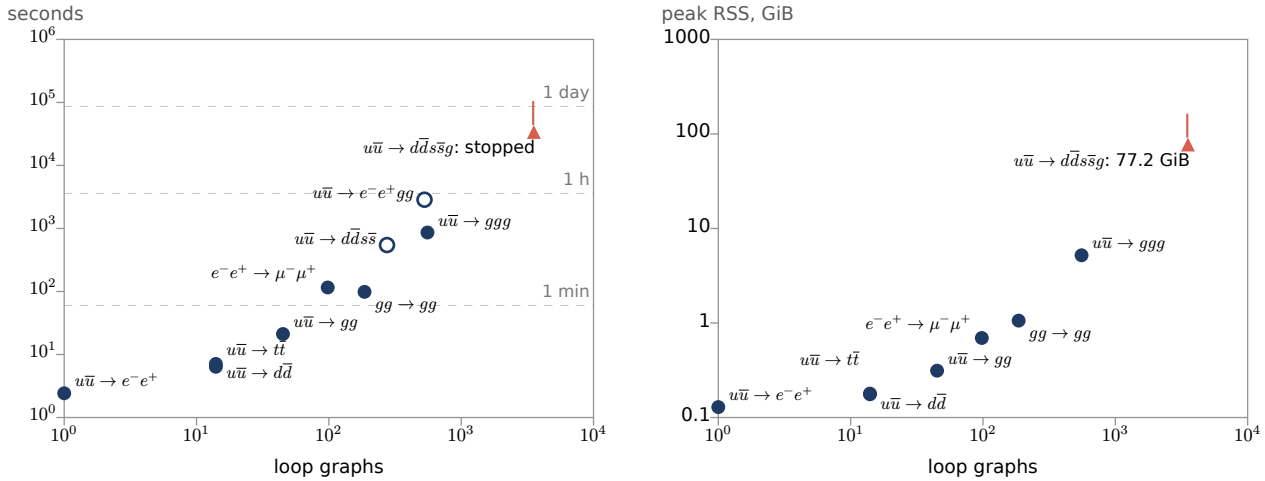
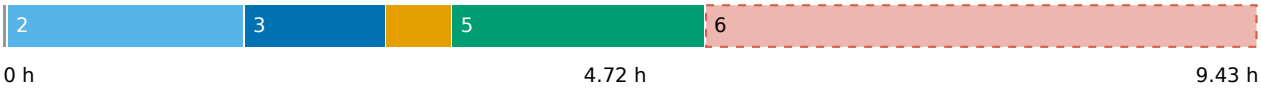


Figure 2: **Generation time and peak memory against loop-graph count.** Filled: the timed processes of Table 2 (fresh process, source capture to complete plan). Open circles: cold generation observed during the complete correctness runs of $u\bar{u} \to e^-e^+gg$ and $u\bar{u} \to d\bar{d}s\bar{s}$ (no memory record). Triangle with tick: the seven-leg control, which ended without a result; its time and memory are lower bounds. Generation grows faster than linearly in the graph count.



	Stage	Interval, h	Duration, s	Max. RSS at end
1	Model and graph generation	0.00–0.04	134	0.674 GiB
2	Recurrence DAGs and colour overlaps	0.04–1.82	6 405	3.49 GiB
3	Born projector	1.82–2.88	3 819	3.49 GiB
4	Dual weights and 751 family projectors	2.88–3.38	1 796	11.3 GiB
5	Counterterm tree enumeration and matching	3.38–5.28	6 852	77.2 GiB
6	UV/R2 projector (unfinished)	5.28–9.43	≥ 14 944	not recorded

Figure 3: **Where the seven-leg generation spent its time.** Stage lines of the eff09b6a correctness run of $u\bar{u} \to d\bar{d}s\bar{s}g$ (64 Born, 3 540 loop graphs, 751 families, CPU 376). The UV/R_2 projector had run for 4.15 h when the process was last seen at 9.43 h; it ended without a result. Memory is the maximum resident set recorded at the end of each stage.

5 Leading colour

Leading-colour (LC) evaluation keeps the highest common power of N_c at fixed N_f (Section 12). Table 5 pairs each LC run with a full-colour run of the same process and R_2 strategy in one separate campaign (revision 95c4ed50, CPU 379); its full-colour times therefore differ from Table 2. Both paths still evaluate every graph, so LC is not yet faster. Next-to-leading colour is validated but not timed.

Table 5: **Leading colour against full colour**, µs per point, binary64, median and min–max of five warm calls. The LC/full cell is coloured with the palette of Table 2; no external provider enters this table. Generation: fresh process to complete plan, LC / full.

Process	R_2	LC, $B = 1$	Full, $B = 1$	LC/full	LC, $B = 128$	Full, $B = 128$	LC/full	Generation, s
$u\bar{u} \to e^-e^+$	model	360 335–458	351 302–440	×1.0	103 97.2–115	98.1 90.2–101	×1.1	1.61 / 1.61
$u\bar{u} \to d\bar{d}$	model	1 640 1 490–1 660	1 540 1 470–1 570	×1.1	609 595–626	647 621–680	×0.9	6.05 / 5.27
$u\bar{u} \to t\bar{t}$	model	1 730 1 670–2 100	1 940 1 790–2 120	×0.9	724 704–735	730 691–743	×1.0	5.64 / 5.25
$gg \to gg$	direct	34 700 33 300–36 100	34 300 33 500–34 500	×1.0	14 600 14 400–16 200	18 800 17 500–20 900	×0.8	127 / 161

The $gg \to gg$ row uses the bounded direct-R_2 control for both colour modes; its full-colour time is the only direct-R_2 observation. No direct-versus-model cost comparison is made, because the model-R_2 row of Table 2 belongs to a different campaign and CPU.

6 Capabilities and open gates

Table 6: **Capability matrix** of the current engine. Sources: Section 16.

Feature	Status	Boundary
Model import	supported	Lossless UFO loop models (Laurent parameters, UV/R_2 categories, loop groups) via the published loader 1.0.1; Goldstone-alias loader PR 3 and a coherent Community package are pending.
QCD interference, massless and external tops	supported	Validated to $2 \to 4$ against MadLoop/Recola in binary64, DD and AP (Table 2, Table 3).
Internal massive quarks	supported	Complete $gg \to t\bar{t}$ (internal top loops, top-mass UV and R_2 two-points), $u\bar{u} \to t\bar{t}$ (top-loop gluon self-energy) and $e^-e^+ \to t\bar{t}h$ (top Yukawa and Goldstone counterterms) match MadLoop in binary64, DD and AP. Not timed.
Model UV and R_2	supported	Consumed from the UFO model for QCD; no completeness check of counterterm matching; tadpoles, one-point counterterms and bivalent mixing unsupported.
Direct R_2	deferred	Deferred by the user; certified bounded controls only (full-colour $gg \to gg$); public dispatch gated.
Coupling orders	supported	Joint QCD/QED eligibility as in MadLoop; every squared order and all three Laurent coefficients returned.
Electroweak	partial	Real-mass $e^-e^+ \to \mu^-\mu^+$ and $e^-e^+ \to u\bar{u}$ (G_μ); 1351 of 1918 G_μ counterterm bases uncertified; no joint QCD+EW.
Restriction cards	supported	Every shipped card of `loop_sm` (incl. `-ckm`), `loop_qcd_qed_sm` and `loop_qcd_qed_sm_Gmu` loads through the production simplification with MadGraph semantics; entries without a model parameter are ignored and recorded, cards MadGraph rejects are rejected.
Complex-mass scheme	open	Matched component controls only; not wired into generation.
General models	partial	Scalar QCD $u\bar{u} \to \sigma_8\sigma_8 g$ with isolated dependency candidates.
Full colour	partial	To six legs; pairwise exact overlaps make seven gluons infeasible.
LC / NLC	partial	Fixed-N_f LC/NLC validated to six legs; formal $\mathrm{SU}(N_c)$ fails on generic seven-leg networks.
Colour FFT	partial	Certified rectangular binary64 full/NLC with exact DD/AP fallback; forced-LC and correlated FFT missing.
Correlations and spin sources	partial	Directed complex colour histories, photon/gluon and W sources; EW/CMS sources and resolved correlated output open.
Flavour mappings	partial	Identity-port species reuse; Bose permutations partial; charge conjugation missing.
Arithmetic	supported	Genuine binary64, DD and AP end to end from exact inputs.
Batching and rescue	partial	Native Rust executor; timings use the Python coordinator; rescue on hard failure only, with no accuracy trigger and two implementations.
Helicity selection	open	All 2^n helicities are evaluated, including structurally zero ones.
Interfaces	partial	Python, TOML/CLI, artifacts, Rust/C/C++/Fortran/UMAMI SDKs; delivered runtime uses the interpreted evaluator; benchmark/profile commands broken; no wheel of the current source.
Required $2 \to 5$ set	open	0 of 8 validated or timed; provider references exist for 6 of 8 (Table 1).
Performance gates	open	No timed process is faster than MadLoop (front page).
NinjaRS and oneloopmaster	partial	Generic reduction up to seven propagators; scalar-master fixes (PR 9, PR 10) not yet adopted.

7 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$, the ordinary observable is

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\left[B_a^\dagger G_{BL} L_{b,k}\right], \quad k \in \{0, -1, -2\},$$

where G_{BL} is the rectangular colour matrix between the Born and loop colour bases and $\mathcal{N}$ holds initial-state averages and identical-particle factors. Couplings are included; nothing is divided by the Born. The coefficients follow the MadLoop convention

$$V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2} \right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

and the Recola controls convert with $V_0^{\mathrm{ML}} = V_0^{\mathrm{Recola}} + \left(\frac{\pi^2}{6}\right) V_{-2}$. Correction eligibility is a property of each graph: a mixed four-quark process with $B = B_{20} + B_{02}$ and $L = L_{40} + L_{22} + L_{04}$ receives both $B_{20}^* L_{22}$ and $B_{02}^* L_{40}$ at squared order $(4, 2)$, and joint QCD+QED selection admits mixed loops that neither single correction contains. Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$, which can be complex.

8 Generation and evaluation pipeline

Figure 4 shows the current dataflow. Generation runs once per process in Python on top of Symbolica and its Community packages; the per-event path runs in a Rust executor. The engine works *per Feynman diagram*: every loop and Born graph owns a coefficient program, colour enters through exact diagram-by-diagram overlaps, and the projections that combine graphs, helicities and colour terms are built as whole-process symbolic expressions.

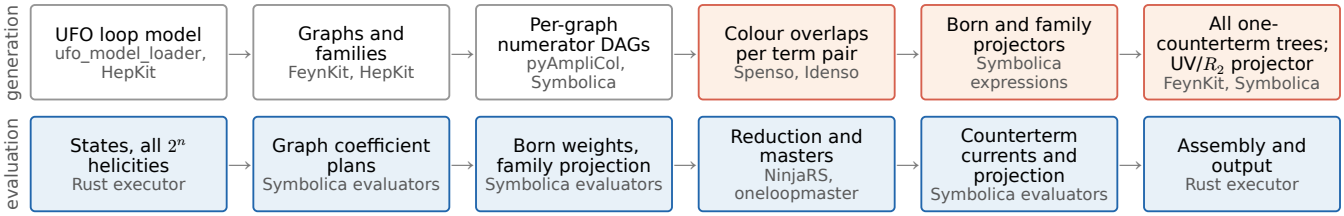


Figure 4: **Current pipeline.** Upper lane: generation, once per process, in Python on Symbolica and its Community packages. Lower lane: evaluation per batch in the Rust executor. Orange: whole-process symbolic constructions that dominate generation (Section 14); blue: native batched numerics. The owning dependency is named under each step.

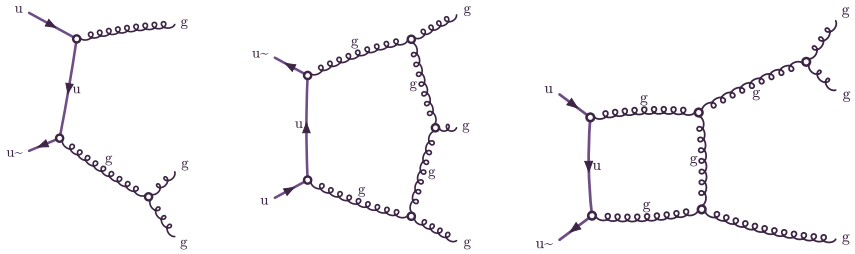


Figure 5: Three graphs of $u\bar{u} \to ggg$, drawn by FeynKit's renderer from its own graph generation: a Born graph, a pentagon, and a box with a tree attachment. The present engine evaluates the attached tree current again inside the program of every loop graph that contains it.

Model. The upstream loader keeps Laurent-valued parameters and couplings, UV and R_2 categories, loop-particle groups and external-field factors. pyAmpliCol prepares one scalar program per coupling, a structural HepKit model and one counterterm rule per vertex, loop group and coupling order. Lorentz and colour tensors remain native Idenso/Spenso objects throughout; admission of a vertex frame is currently by an explicit list of certified structures.

Graphs. FeynKit generates Born and one-loop graphs with its own filters disabled; pyAmpliCol applies MadLoop-equivalent physical rejection and assigns each graph to an integral family through HepKit's family mappings, keeping routed propagators, bridges and the exact routing transformation.

Table 7: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals, and owns the process-level assembly.

Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, series and coefficient extraction; compiled evaluators (interpreted, SymJIT in binary64); exact rationals and arbitrary precision
HepKit (Community)	Structural model; integral families and family mappings
FeynKit (Community)	Born and loop graph generation, momentum routing, symmetry factors, counterterm trees, graph drawing (Figure 5)
Spenso, Idenso (Community)	Lorentz and colour tensors, gamma and colour algebra, exact colour overlaps
NinjaRS	Integrand reduction with Laurent expansion and rank admission
oneloopmaster	Scalar master integrals in binary64, DD and AP
pyAmpliCol	Order selection and physical rejection, coefficient programs, Born weighting, counterterm matching, projections, assembly, Rust executor, artifacts and interfaces

9 Numerators, families and Born weighting

Each graph's loop numerator is a polynomial in the loop momentum q. A current holds the coefficients of that polynomial; a local vertex tensor T acting on the previous current J gives the next one, $J' = T \cdot J$, with coefficient extraction prepared once symbolically and evaluated natively in 16-row tiles. Every graph owns its own coefficient program, so tree attachments and open-loop segments shared by many graphs are recomputed in each (Figure 5).

The one decision that already scales well is to weight by the Born *before* reduction. For each point and helicity h, the complex weight ω_{gh} of graph g is the component of the Born row $B_h^\dagger G$ that belongs to the colour structure of g, and each family F is reduced once:

$$B^\dagger G L = \sum_F \int d^d q \, \frac{\sum_{g\in F} \sum_h \omega_{gh} N_{gh}(q)}{\prod_{i\in F} D_i(q)}.$$

One reduction per point, coupling sector and family replaces one per graph, helicity and colour term; Table 4 shows reduction plus masters at 0.4–3.9 % of the clocked time. Weights stay complex until the directed bilinear is formed, because colour histories and other off-diagonal observables need both cross terms.

10 Reduction and scalar masters

NinjaRS receives the routed denominators, the weighted polynomial coefficients and exact invariants in the carrier's precision; weighted rank counts μ^2 twice and explicit ε sectors combine by Laurent multiplication. The reducer owns rank admission (up to rank $n+2$) and reduces up to seven propagators generically, including a rank-eight seven-propagator control that cancels exactly to a triangle. oneloopmaster evaluates the scalar masters natively in all three arithmetics; its complex-mass and high-precision branch fixes (PR 9, PR 10) pass isolated controls but are not yet adopted.

11 Counterterms and rational terms

The production strategy uses the UV and R_2 effective vertices of the UFO model. Every tree with exactly one counterterm vertex is enumerated by FeynKit and matched against the admitted loops, keeping attachment currents, ordered legs, coupling orders and loop-species alternatives; exact vertex transposition identifies Bose-equivalent slots. Local UV, external-field factors and R_2 remain separately inspectable before joining the same ordered interference. Model R_2 currents already carry the loop normalisation. A symbolic projector over helicities, currents and Born terms then contracts all insertions.

Direct R_2, an evanescent recurrence with numerator $N_0 + \varepsilon N_1 + \varepsilon^2 N_2$ and reduced sectors $\left(F_j, S_j, D_j\right)$ giving $V_0 = F_0 + S_1 + D_2$, $V_{-1} = S_0 + D_1$, $V_{-2} = D_0$, is certified on bounded vectorlike and Yang–Mills controls only and is deferred.

12 Colour, correlations and mappings

Full colour contracts exact Born and loop colour tensors through Spenso/Idenso into the rectangular G_{BL}, one overlap per pair of diagram colour terms, with no numerical colour fits. For a certified formal-N_c model, LC and NLC keep the common highest power P across all contributions of a squared order:

$$\mathrm{LC} : N_c^P, \quad \mathrm{NLC} : N_c^P, N_c^{P-1}, N_c^{P-2}, \quad N_f \text{ fixed}.$$

Directed correlations act with native charge tensors on batched shared currents. Identity-port flavour members reuse an exact plan; certified rectangular FFT contraction covers binary64 full and NLC colour.

13 Precision, batching and rescue

Binary64, DoubleFloat and arbitrary precision run through coefficients, kinematics, reduction, masters and counterterms without promotion through binary64; decimal inputs and rational model coefficients are exact until the carrier is chosen. Ordered layouts preserve point, helicity and event identities through tails and compaction. Rescue reevaluates a failed point in higher precision and accepts it when two higher results agree on every component; there is no accuracy-based trigger yet, and the Python and Rust coordinators implement rescue separately.

14 Measured scaling walls

The per-diagram design meets the walls of Table 8, which together put the required $2 \to 5$ processes out of reach. Values are from the seven-leg stage log (Figure 3) and from stack samples and probes of the 2026-10-20 assessment (Section 16).

Table 8: **Scaling walls of the current engine.**

Wall	Measured	Consequence
Symbolic whole-process projectors (Born, families, UV/R_2), unrolled over helicities	Seven legs: Born projector 3 819 s; family projectors 1 796 s; UV/R_2 projector unfinished after 4.15 h. Family and counterterm projectors take 90.5 % and 8.4 % of the 394 MB six-leg artifact. Measured with the former quadratic term accumulation; one-pass sums now build the six-leg $u\bar{u} \to d\bar{d}s\bar{s}$ counterterm projector expression in 2.2 s instead of 85.6 s.	Blocks every $2 \to 5$ process; recompiled per colour mode, spin class and correlation.
Enumeration of all one-counterterm trees, then matching to loops	$u\bar{u} \to d\bar{d}s\bar{s}$: 50 s, 832 MB; $gg \to ggg$: 86 s, 1.28 GB; $gg \to gggg$: over 8 GB within 165 s; seven legs: 6 852 s up to 77.2 GiB.	Blocks $gg \to gggg$ and above; dominates memory.
Pairwise exact colour overlaps, no colour basis	About 5,000 s of the seven-leg DAG stage; formal $\mathrm{SU}(N_c)$ fails on 14 of 24 random seven-gluon pairs.	Seven-gluon full colour infeasible; no LC/NLC at seven legs.
Per-diagram numerators without sharing	54 % of loop-node and 95 % of counterterm-node evaluations are duplicates; numerator plus Born weighting dominate Table 4.	Runtime ×2.0–×14.8 MadLoop (front page).
No helicity selection	Structurally zero helicity rows: Drell–Yan 12 of 16, $u\bar{u} \to d\bar{d}s\bar{s}$ 56 of 64.	Two- to eightfold waste in every stage, squared in the projector builders.

15 Redesign in progress

The per-diagram numerator core is being replaced. The replacement is at the design stage and is not implemented; nothing in this report measures it. Its centre is a shared open-loop recursion in the style of OpenLoops and Recola, built on pyAmpliCol's tree engine and its colour-flow currents, so that tree attachments and open-loop segments are computed once per process rather than once per graph. Table 9 lists the planned answer to each wall. Model import, counterterm rules, FeynKit graphs and families, Born weighting before reduction, the NinjaRS and oneloopmaster adapters, the exact carriers, the Rust executor and the public interfaces are kept.

Table 9: **Planned replacements** (design status; resumption assessment §4).

Current construction	Planned replacement	Needed for
Symbolic whole-process projectors	Native sparse contractions over a helicity row axis	Every $2 \to 5$ generation; artifact size
All one-counterterm trees	Enumeration of counterterm sites, with a completeness check	$gg \to gggg$ and above; memory
Pairwise colour overlaps	Canonical overlap cache, then a fixed trace or colour-flow basis	Seven-gluon full colour; LC/NLC at seven legs
Per-diagram numerator programs	Shared open-loop recursion on the tree engine	Speed gates; $gg \to gggg$, $gg \to ggggg$, $u\bar{u} \to ggggg$
All 2^n helicities	Selection of non-vanishing helicities	Two- to eightfold saving in every stage
Listed counterterm structures	Generic tensor lowering for admission; CMS as a model transformation	Both $t\bar{t}$ targets; EW and CMS

16 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Timings:** `data/measurements.json` (samples, stages, provider references), `data/provider-cold.json` (provider generation and setup), `data/colour-measurements.json` (LC/full) and `data/provider-audit.json` (Recola Drell–Yan and OpenLoops audit). Their drivers are not yet in the repository: `workspace/experiments/loop_live_*.py` and `workspace/reference/live_provider_benchmark.py`.
- **Process status:** `data/processes.json`, from the plan's §5.7, the continuation handoff (§3, §6, §7) and the 2026-10-20 resumption assessment (§1, §2.3, §8), which is also the source of the probe and stack-sample values in Table 8. Required-process provider references: `tests/fixtures/loop/required/*.json`. Case labels, missing-value markers and the release-build A/B (`workspace/reports/release-build-ab.md`) are also recorded there.
- **Generation:** `data/generation.json`, written by `scripts/collect_generation.py` from the retained correctness-run records and the seven-leg stage log. Graph drawings: `scripts/draw_graphs.py`.
- **Conventions:** MadLoop standalone, Recola, OpenLoops on-the-fly reduction, four-dimensional formulation.